

Manipulação de Arquivos de Texto

Pré-requisitos: variáveis e tipos, laços de repetição, listas, dicionários, conjuntos (sets), tratamento de exceções (Aula 08). Projeto de fundo: E-commerce simples.

1. Material explicativo

1.1 Por que manipular arquivos de texto?

Até agora, todos os dados dos nossos programas "morriam" quando o programa era encerrado: catálogo, notas, listas etc. só existiam na memória RAM enquanto o script rodava. Para que os dados sejam **persistentes** (ou seja, continuem existindo depois que o programa terminar) precisamos gravá-los em algum lugar — o mais simples é um **arquivo de texto**.

Arquivos de texto também são muito usados para:

- Trocar dados entre programas diferentes (ex.: uma planilha exporta um `.txt` e o Python lê esse arquivo);
- Guardar logs (registros) do que aconteceu durante a execução de um programa;
- Servir como uma "entrada em lote" de dados, evitando digitar tudo de novo pelo teclado.

1.2 Abrindo um arquivo: a função `open()`

Antes de ler ou escrever em um arquivo, é preciso abri-lo com a função embutida `open()`. Ela recebe (pelo menos) dois parâmetros posicionais: o caminho/nome do arquivo e o modo de abertura.

```
python

arquivo = open('produtos.txt', 'r')
# Abrindo 'produtos.txt' em modo de leitura ('r')
```

Se o arquivo estiver na mesma pasta do script, basta informar o nome. Caso contrário, é necessário informar o caminho completo (ex.: `'C:\\dados\\produtos.txt'` ou `'/home/usuario/dados/produtos.txt'`).

1.3 Modos de abertura

Modo Uso

<code>r</code>	Abre um arquivo já existente para leitura . Se o arquivo não existir, gera <code>FileNotFoundError</code> .
<code>w</code>	Abre para escrita . Se o arquivo já existir, seu conteúdo é apagado . Se não existir, é criado.
<code>a</code>	Abre para escrita ao final (append). Se o arquivo já existir, o conteúdo é preservado e o que for escrito entra depois dele. Se não existir, é criado.
<code>r+</code>	Leitura e escrita, a partir do início do arquivo.
<code>w+</code>	Escrita e leitura, apagando o conteúdo existente (ou criando o arquivo).
<code>a+</code>	Escrita (ao final) e leitura.
<code>x</code>	Cria um novo arquivo para escrita; falha (gera erro) se o arquivo já existir.

Também é possível combinar `b` (binário) ou `t` (texto, que é o padrão) ao modo, por exemplo `'rb'`. Nesta aula trabalhamos apenas com arquivos de **texto**, então não precisamos nos preocupar com o modo binário.

1.4 Lendo o conteúdo de um arquivo

Existem várias formas de ler um arquivo já aberto:

```
python

# lê o arquivo inteiro de uma vez, como uma única string
texto = arquivo.read()

# lê apenas os primeiros 20 caracteres
trecho = arquivo.read(20)

# lê uma única linha (até encontrar a quebra de linha '\n')
linha = arquivo.readline()

# lê todas as linhas e devolve como uma lista de strings
linhas = arquivo.readlines()
```

Como o objeto arquivo é **iterável**, também é muito comum percorrê-lo linha a linha com um `for`, sem precisar chamar nenhum desses métodos:

```
python
```

```
for linha in arquivo:
    print(linha)
```

Cada `linha` lida dessa forma vem com o `\n` (quebra de linha) no final, exceto possivelmente a última linha do arquivo. Por isso é comum usar `linha.strip()` ou `linha.rstrip('\n')` para remover essa quebra antes de processar o conteúdo, e `linha.split(separador)` para quebrar a linha em campos (assunto já visto na Aula 05 — Manipulação de Strings).

1.5 Escrevendo em um arquivo

Para escrever, o arquivo precisa ter sido aberto em um modo que permita isso (`w`, `a`, `x`) ou as variações com `+`:

```
python

arquivo.write('Esta linha será escrita no arquivo\n')

# escrevendo várias linhas de uma vez, a partir de uma lista
arquivo.writelines(['Camiseta\n', 'Calça\n', 'Boné\n'])
```

Repare que `write()` e `writelines()` **não** adicionam `\n` automaticamente — se você quiser que cada informação fique em uma linha separada, é necessário incluir o `\n` você mesmo.

1.6 Fechando um arquivo

Todo arquivo aberto com `open()` deve ser fechado com `close()` quando não for mais necessário, liberando o recurso do sistema operacional:

```
python

arquivo.close()

if arquivo.closed:
    print('Arquivo fechado.')
```

Esquecer de fechar um arquivo pode causar problemas, como dados que ainda não foram gravados fisicamente em disco (ficam apenas em um "buffer" de memória) ou o arquivo ficar bloqueado para outros programas.

1.7 Tema complementar: a instrução `with`

Como fechar arquivos manualmente é fácil de esquecer — principalmente se o programa lançar uma exceção no meio do caminho —, o Python oferece a instrução `with`, que garante o fechamento automático do arquivo assim que o bloco indentado terminar (mesmo que ocorra um erro dentro dele):

python

```
with open('produtos.txt', 'r', encoding='utf-8') as arquivo:
    for linha in arquivo:
        print(linha.rstrip())

# Aqui fora do bloco with, o arquivo já foi fechado automaticamente.
```

O `with` é chamado de **gerenciador de contexto** e é usado sempre que precisamos garantir a liberação de algum recurso (arquivos, conexões de banco de dados, etc.). A partir de agora, vamos **preferir sempre o `with`** em vez de abrir e fechar manualmente com `open()`/`close()`.

É possível abrir mais de um arquivo no mesmo `with`, separando-os por vírgula:

python

```
with open('entrada.txt', 'r', encoding='utf-8') as arq_entrada, \
     open('saida.txt', 'w', encoding='utf-8') as arq_saida:
    for linha in arq_entrada:
        arq_saida.write(linha.upper())
```

Exercícios complementares (tema: instrução `with`)

Exercício complementar 1 — Usando a instrução `with`, abra o arquivo `notas.txt` (uma linha por aluno, no formato `RM,NOME,NOTA1,NOTA2,NOTA3,NOTA4`) e imprima apenas os nomes dos alunos, um por linha.

Exercício complementar 2 — Usando `with` para abrir dois arquivos ao mesmo tempo (`ips.txt` para leitura e `ips_maiusculo.txt` para escrita), grave no segundo arquivo cada linha do primeiro em letras maiúsculas.

1.8 Tratando caracteres especiais: o parâmetro `encoding`

Nomes com acentos ou caracteres especiais (ex.: "José", "Não") podem ser lidos ou escritos incorretamente dependendo da codificação padrão do sistema operacional. Para evitar esse problema, é uma boa prática **sempre informar** `encoding='utf-8'` ao abrir arquivos de texto:

python

```
with open('alunos.txt', 'r', encoding='utf-8') as arquivo:
    for linha in arquivo:
        print(linha)
```

1.9 Tema complementar: tratando erros ao acessar arquivos

Na Aula 08 aprendemos `try`/`except`/`else`/`finally`. Esse conhecimento é extremamente útil ao trabalhar com arquivos, já que tentar abrir um arquivo que não existe (em modo `'r'`) gera a exceção `FileNotFoundError`. Em vez de deixar o programa parar com um erro, é possível capturar essa exceção e tratar a situação de forma amigável:

```
python

nome_arquivo = input('Nome do arquivo a abrir: ')
try:
    with open(nome_arquivo, 'r', encoding='utf-8') as arquivo:
        print(arquivo.read())
except FileNotFoundError:
    print(f"Erro: o arquivo '{nome_arquivo}' não foi encontrado.")
```

Repare que o `with` continua sendo usado normalmente **dentro** do `try` — as duas ferramentas se combinam sem nenhum conflito: o `with` garante o fechamento do arquivo caso ele tenha sido aberto com sucesso, e o `try`/`except` trata o caso em que a abertura falha.

Exercícios complementares (tema: tratamento de erros em arquivos)

Exercício complementar 1 — Peça ao usuário o nome de um arquivo e tente abri-lo em modo leitura. Trate a exceção `FileNotFoundError`, exibindo uma mensagem amigável caso o arquivo não exista.

Exercício complementar 2 — Crie uma função chamada `ler_arquivo_seguro` que recebe o caminho de um arquivo, tenta ler todas as suas linhas (retornando-as em uma lista, sem o `\n` do final) e, caso o arquivo não seja encontrado, imprime uma mensagem de erro e retorna uma lista vazia.

2. Exercícios principais (dificuldade crescente)

Exercício 1 — Solicite ao usuário 10 números inteiros e armazene-os em um arquivo de texto chamado `numeros.txt` (um número em cada linha).

Exercício 2 — Abra o arquivo `numeros.txt` criado no exercício anterior. Leia o conteúdo do arquivo e mostre o somatório de todos os números contidos nele.

Exercício 3 — Faça um programa que crie um arquivo de texto denominado `arquivo.txt` e permita que o usuário grave diversos caracteres nesse arquivo até que seja digitado o caractere "0" (zero).

Exercício 4 — Solicite ao usuário diversos números inteiros (até que seja digitado o número zero). Armazene os números pares em um arquivo (`pares.txt`) e os números ímpares em outro (`impares.txt`).

Exercício 5 — Faça um programa que abra os dois arquivos criados no exercício anterior e copie-os para um novo arquivo (`numeros_ordenados.txt`), colocando-os em ordem crescente.

Exercício 6 — O arquivo (`notas.txt`) armazena, em cada linha, o RM, o nome e quatro notas de um aluno, separados por vírgula (`RM,NOME,NOTA1,NOTA2,NOTA3,NOTA4`). Leia esse arquivo e calcule a média de cada aluno.

3. Exercícios complementares

Estes exercícios usam arquivos de dados fornecidos em aula (`ips.txt`, `notas.txt`, `foods.txt` e `race.txt`) e são um pouco mais desafiadores que os principais — por isso são marcados como **complementares**.

Exercício complementar 1 — O arquivo (`ips.txt`) traz uma listagem de endereços IP que acessaram um site (um IP por linha, podendo haver repetidos). A partir desse arquivo, gere um novo arquivo (`ips_unicos.txt`) com uma listagem de IPs únicos (sem valores repetidos).

Exercício complementar 2 — Usando o arquivo (`notas.txt`) (`RM,NOME,NOTA1,NOTA2,NOTA3,NOTA4`), gere dois novos arquivos: (`aprovados.txt`), contendo (`RM,NOME,MEDIA`) dos alunos aprovados (média igual ou superior a 6.0), e (`reprovados.txt`), contendo (`RM,NOME,MEDIA`) dos alunos reprovados.

Exercício complementar 3 — O arquivo (`foods.txt`) traz, em cada linha, (`NOME,ID,ALIMENTO_FAVORITO`). Escreva um programa que leia esse arquivo e informe o alimento preferido pela maioria das pessoas.

Exercício complementar 4 — O arquivo (`race.txt`) traz dados de 10 pilotos de carros elétricos: nome, número do carro, tempo de volta (s), velocidade média (km/h), consumo médio de energia (kWh), temperatura média do motor (°C) e posição final. Calcule o consumo médio de energia de todos os carros, identifique o piloto com o menor consumo de energia e o piloto com o maior tempo de volta.

4. Implementação no projeto de fundo (E-commerce simples)

Enunciado: aplique manipulação de arquivos de texto no projeto, **sem alterar** nenhuma função já existente (`cadastrar_produto`, `atualizar_estoque`, `buscar_produtos_por_termo`, `buscar_produtos_por_tags`, `obter_todas_tags`, `exibir_catalogo`, `vender_produto` e as exceções da Aula 08 continuam exatamente como estão). Adicione três novas funções a (`ecommerce.py`):

1. `exportar_catalogo_para_arquivo`: grava o catálogo atual em um arquivo de texto, um produto por linha (`codigo;nome;preco;estoque;tags`).
2. `importar_produtos_de_arquivo`: lê um arquivo de texto com produtos no formato (`nome;preco;estoque`) (um por linha) e cadastra cada um deles no catálogo,

reaproveitando a função `cadastrar_produto` já existente. Se o arquivo não existir, trate o erro (tema complementar desta aula) em vez de deixar o programa quebrar.

3. `gerar_relatorio_estoque_baixo`: gera um arquivo de texto com a lista de produtos cujo estoque está abaixo (ou igual) de um limite informado.